

老系统如何升级编程模式 — AI全栈开发工程师能力展示系统

- 项目定位：**面向大量基于Spring生态甚至更老的Java生态的存量IT项目，探讨如何升级到AI-first、Agent智能编程模式，提升需求响应速度与可维护性，更好适应企业业务发展。
- 切入点：**以一个典型且常见的**CMS内容管理系统**为Demo载体，展示从架构设计、技术栈选型到AI智能编程模式升级的完整能力模型。
- 开发工具：**Codex 全程AI协作开发
- 工期：**约47小时，18个里程碑迭代交付
- 项目地址(私仓)：**<https://github.com/wind-laughing/traffic-demo>

一、背景与问题

老项目面临的真实困境

大量企业的Java项目以Spring生态（甚至更老的Struts/SSH）为基础沉淀多年，面临一系列积重难返的问题：

问题	表现
需求响应慢	一个看似简单的需求变更，从分析设计到开发测试上线，周期以周计
可维护性差	代码耦合严重、文档缺失、依赖老旧、不敢升级版本
人员依赖重	核心模块只有一两个人能改，其他人不敢动，离职即断档
测试覆盖低	缺乏自动化测试，每次上线靠"人肉回归"
交付质量波动	依赖个人经验而非制度保障，换个人代码风格和安全意识全变

这些问题不是技术债的问题，是**编程模式的问题**。不是代码写坏了，是用老的方式写代码已经跟不上业务的变化速度。

AI-first编程模式为什么能解决

AI编码工具（Codex/Claude Code等）不是"帮程序员写得快一点"——它是**重新定义了人和代码之间的关系**：

- AI出初稿 → 人做判断和决策 → 门控验收 → 沉淀为可复用资产
- 复杂不再等于慢：一个8微服务分布式系统，60分钟出架构，47小时含休息完成18个迭代，每一版都跑通可运行
- 人不盯代码，盯架构、盯质量、盯方向——从**"写代码的人"变成"判断代码的人"**

但这里有一个关键前提：**AI编码工具不是万能的**。它是一个强大但无知的执行者——它需要知道老系统的技术生态、架构约束、历史包袱，才能给出可行的方案，而不是"推倒重来"的理想方案。

这就引出了对AI全栈工程师的能力要求。

二、AI全栈工程师的能力模型

一个合格的AI全栈工程师，不只是会用AI工具，还需要具备以下能力层次：

能力维度	说明
老系统理解	熟悉Spring生态乃至更老的Java技术栈，理解分层架构、事务控制、权限模型、前后端交互模式——知道老代码为什么写成那样，而不是一上来就"重构"
技术选型判断	知道什么场景该用微服务、什么场景单体就够了；什么场景上MQ、什么场景同步调用可以——不是"我会用"而是"该不该用"
AI工具链掌握	熟练使用Codex/Claude Code等AI编码工具，理解其能力边界和常见陷阱，能设计有效的Prompt策略和Agent角色分工
架构设计能力	能根据业务现状设计合理的系统架构，不追求技术先进性，追求"对企业当前阶段最合适的方案"
工程化思维	建立规约体系、质量门禁、版本管理、文档沉淀——不走新项目新规范、老项目老规矩的老路
团队赋能意识	不是一个人用AI把活干完就算了，而是把AI工具、Skill、规约变成团队可复用的资产，带动团队整体升级

本系统的设计目标，就是用**一个典型的CMS场景**，具象化展现这套能力模型。

三、为什么选CMS做切入点

CMS（内容管理系统）是Java生态中最具代表性的应用场景之一：

- **业务复杂度适中**：CRUD + 状态流转 + 权限控制 + 审批流程 + 消息通知——覆盖一个业务系统的核心逻辑，又不至于太庞大
- **技术栈覆盖全面**：需要后端框架、数据库、缓存、消息队列、前端交互、权限模型——几乎能验证Java全栈的所有常见技术点
- **老项目代表性**：几乎每一家做Java的企业都接触过CMS或类似的管理后台系统，容易理解、容易对比

同时，CMS天然适合展示**从传统模式到AI-first模式的升级路径**：

对比维度	传统模式	AI-first模式
架构设计	人类查阅多个竞品+经验判断，1-3天	AI协助生成架构方案+多方案对比，60分钟出初版
编码实现	按模块逐行编写，联调发现问题再回头修	AI出初稿+人类审查+联调，每轮迭代都跑通
问题排查	GPT分步贴代码碰运气	结构化问题输入+Agent独立调试+陷阱根因记录
资产沉淀	只有代码和注释，靠人传人	Skill/Command/质量门禁/陷阱档案，团队可复用

四、本系统展示的能力

以下通过本系统的设计与开发过程，逐项回应对AI全栈工程师的能力要求：

能力一：老系统理解 + 技术选型判断

本系统采用的技术栈，全部是当前Spring生态的主流选择，也是大量老系统正在或将要升级的路线：

层级	技术	定位说明
后端语言	Java 17	当前主流长期支持版本，从Java 8迁移的标准目标
后端框架	Spring Boot 3.x + Spring Cloud	Spring生态核心，老系统升级的主流方向
ORM	MyBatis Plus	MyBatis的增强版，大量老系统的直接升级路径
注册中心	Nacos	相比Eureka/Consul，更轻量且支持配置中心二合一
数据库	MySQL 8 + PostgreSQL 16	双数据源展示多库协同能力（老系统常见场景）
缓存	Redis	企业级标配，无需解释
消息队列	RabbitMQ	最成熟的MQ之一，学习成本低、生态完善
前端	Vue 3 + Element Plus	当前Spring Boot项目最常见的前后端分离方案

不是炫技。 每一个技术选型都有明确的"这是老项目升级的主流路线"的底层逻辑。

能力二：AI工具链掌握

全程使用Codex（GPT-5）作为AI协作编码工具。不是"用AI代写代码"的一锤子买卖，而是沉淀了一整套AI辅助开发的机制体系：

沉淀资产	数量	说明
AI Agent 定义	4个	全能/产品/QA/架构四角色
AI Skill	20+个	可复用工作流程
AI Command	4个	/help /status /save /check
编码陷阱记录	15+条	UTF-8/YAML/MyBatis等踩坑档案
质量门禁	6道	每里程碑强制执行的质检关卡

这些资产的价值在于：**它们是可以复用的。** 换一个项目、换一个团队，同样的Skill和门禁体系可以直接平移。

能力三：架构设计能力

8个微服务的体系设计——Gateway统一鉴权路由、Nacos注册中心、RabbitMQ消息异步解耦、Redis缓存同步、PostgreSQL全文检索——不是拍脑袋决定的，是对CMS业务场景分析后做出的合理判断：

- Gateway集中处理鉴权 → 避免每个服务重复实现
- 服务按业务域拆分 → 各自独立迭代，不影响其他模块
- MQ异步解耦 → 内容发布和消息通知不互相阻塞
- 双数据库 → 适应业务数据和全文检索引擎的不同特性

能力四：工程化思维

系统配套完整的三层工程规约体系：

总纲（做什么）→ 规约（按什么规矩做）→ 方法论（怎么做）

配套Git规约（Conventional Commits）、代码规约、API规约、数据规约、文档规约——体现的不是代码堆砌，而是**有制度可循的工程化思维**。

每个里程碑强制执行6道质量门禁：

阶段	检查项
Phase 2.5	预飞检查（JDK/Maven/编码/超时/目录）
Phase 3.0	Q1-Q5质检（SQL/YAML/MyBatis/SpringBoot/DemoData）
Phase 3.5	编译验证 + 状态更新 + 技术债同步
Phase 4	集成验证（Gateway JWT + Auth端到端）
Phase 5.5	门控 - 等待用户明确确认
Phase 6	收尾归档（Wiki同步/技术债闭环/Commit+Tag）

五、微服务架构

服务	端口	职责
gateway-service	8800	API网关（JWT鉴权 + 路由转发 + CORS跨域）
auth-service	8101	认证中心（登录/注册/JWT签发）
system-service	8102	系统管理（用户/角色/菜单/组织CRUD + RBAC四级权限）
cms-service	8103	内容管理（文章/分类/标签CRUD + 状态机）
knowledge-service	8104	知识库（文档上传/管理/PostgreSQL全文检索）
workflow-service	8105	流程审批（模板管理/审批流转/待办/历史）
notification-service	8106	消息通知（RabbitMQ消费/站内信推送）
ai-service	8107	AI服务（续写/总结/改写 + Mock/Real热切换）

8个微服务全部注册到Nacos，Gateway统一路由转发，形成完整的微服务治理体系。

六、核心业务模块

模块一：权限管理（RBAC四级模型）

用户 > 角色 > 菜单 > 组织，四级权限体系。Spring Security + JWT + 前端路由授权 + 按钮级权限控制，完整权限闭环。

模块二：内容管理（CMS核心）

文章CRUD + 富文本编辑器（TinyMCE）+ 状态机（草稿→待审→已发布→下架）+ 多维度检索（分类/标签/关键词/日期）。这是CMS业务闭环的核心。

模块三：知识库与文档管理

文档上传（PDF/Word/TXT）、树形文件夹目录、PostgreSQL全文检索。

模块四：流程审批引擎

自研轻量工作流引擎（状态机+责任链模式），模板定义→内容提交审核→多节点审批流转→待办/历史。

模块五：消息通知

内容发布/审批通过后RabbitMQ异步通知 → 站内信推送，端到端异步消息闭环。

模块六：AI写作辅助（完整集成）

编辑器内AI续写/总结/改写——调用DeepSeek API，支持Mock/Real热切换（无API Key时可演示Mock数据）。

模块七：AI全栈工程师能力展示

系统内置面试演示入口（/interview），含AI助手问答、架构图可视化、JD对标矩阵、基础设施实时面板、生产管线5个Tab。

AI助手

Galaxy CMS

Dashboard

系统管理

内容管理

文章列表

分类管理

标签管理

知识库

流程审批

通知

AI全栈工程师

AI全栈工程师

系统管理员退出

AI 助手

系统架构

JD 对标

基础设施

生产管线

问任何关于本系统架构、技术栈、AI实践、开发流程的问题

RealMockReal

你 这个项目的亮点是什么？

项目亮点（面试场景）
一、能力证明方式创新
可运行系统 = 面试作品集
与传统简历只说“会什么”不同，本项目直接以**可运行的 CMS 系统**作为能力证明。面试官可以现场登录、操作、查看代码——技术落地一目了然。
二、全栈技术栈闭环
在一个 Demo 中**串联打通**了 JD 要求的全部技术栈：
层次	技术	证明点
系统的技术栈是怎样的？ 项目采用了什么AI开发模式？ 有哪些微服务和数据存储？ 质量门禁机制是什么？ 如何启动整个系统？ 这个项目的亮点是什么？
输入你的问题... 提问
参考来源： project-information.md project-specification.md ai-first-team-sop.md build-parameters.md powershell-encoding-trap.md frontend-api-alignment.md yaml-spring-nesting.md CLAUDE.md

系统架构

Galaxy CMS

Dashboard

系统管理

内容管理

文章列表

分类管理

标签管理

知识库

流程审批

通知

AI全栈工程师

AI全栈工程师

系统管理员退出

AI 助手

系统架构

JD 对标

基础设施

生产管线

Galaxy CMS 系统架构

Gateway 8800

Nacos 注册中心

Auth 8101

System 8102

CMS 8103

AI 8107

Knowledge 8104

Workflow 8105

Notify 8106

MySQL 8
galaxy_cms

PostgreSQL 16
全文搜索 GIN 索引

Redis 7
缓存服务

RabbitMQ

Nacos 2.3.2

WSL2 Ubuntu

Vue 3 + Element Plus 管理后台

前端 8080 | 后端 8080服务 | 双数据库 | MQ + Redis | Nacos 服务发现

技术栈：Java 17 + Spring Boot 3.2 + Spring Cloud 2023 + MyBatis Plus + Vue 3 + Element Plus + MySQL + PostgreSQL + Redis + RabbitMQ + Nacos

JD对标

Galaxy CMS

Dashboard

系统管理

内容管理

文章列表

分类管理

标签管理

知识库

流程审批

通知

AI全栈工程师

AI全栈工程师

系统管理员

退出

AI 助手

系统架构

JD 对标

基础设施

生产管线

JD 要求	DEMO 体现	验证位置
Spring Cloud 微服务架构	8 微服务 + Gateway + Nacos 服务发现	架构图 / Nacos 控制台 :8848
Java 17 + Spring Boot 3	所有后端服务使用 Java 17 + Spring Boot 3.2	各服务 pom.xml / 启动日志
MyBatis Plus 数据库操作	@MapperScan + LambdaQueryWrapper + 逻辑删除	各 Mapper.java / 用户管理页面
MySQL + PostgreSQL 双数据源	MySQL 主库 + PostgreSQL 全文搜索 (GIN)	知识库文档上传/搜索
Redis 缓存	Spring Data Redis (RedisTemplate)	基础设施面板 (M17)
RabbitMQ 消息队列	文章审核通过 -> MQ -> 站内信通知	审批文章 -> 查看通知
Vue 3 + Element Plus	完整管理后台, 19 页面 + 路由守卫 + JWT 状态管理	前端页面整体
Nacos 服务发现与注册	8 服务注册 + Gateway lb:// 路由	Nacos 控制台 / 启动日志
AI 编程实践 (Codex)	全程 AI-first 开发 + 6 个质检 skill + 4 篇经验 wiki	AI 助手 /.claude/skills/
AI 写作集成	富文本编辑器 + AI 续写/总结/改写	内容管理 -> 文章编辑
CI/CD 流水线	GitHub Actions 自动构建	GitHub Actions badge
WSL2 基础设施	WSL2 Ubuntu 运行 MySQL/PG/Redis/RabbitMQ/Nacos	基础设施面板 (M17)

基础设施

Galaxy CMS

Dashboard

系统管理

内容管理

文章列表

分类管理

标签管理

知识库

流程审批

通知

AI全栈工程师

AI全栈工程师

系统管理员退出

AI 助手

系统架构

JD 对标

基础设施

生产管线

✔ Nacos

localhost:8848

✔ MySQL

localhost:3306

✔ PostgreSQL

localhost:5432

✔ Redis

localhost:6379

✔ RabbitMQ

localhost:5672

✔ Gateway

localhost:8800

✔ Auth

localhost:8101

✔ Frontend

localhost:8080

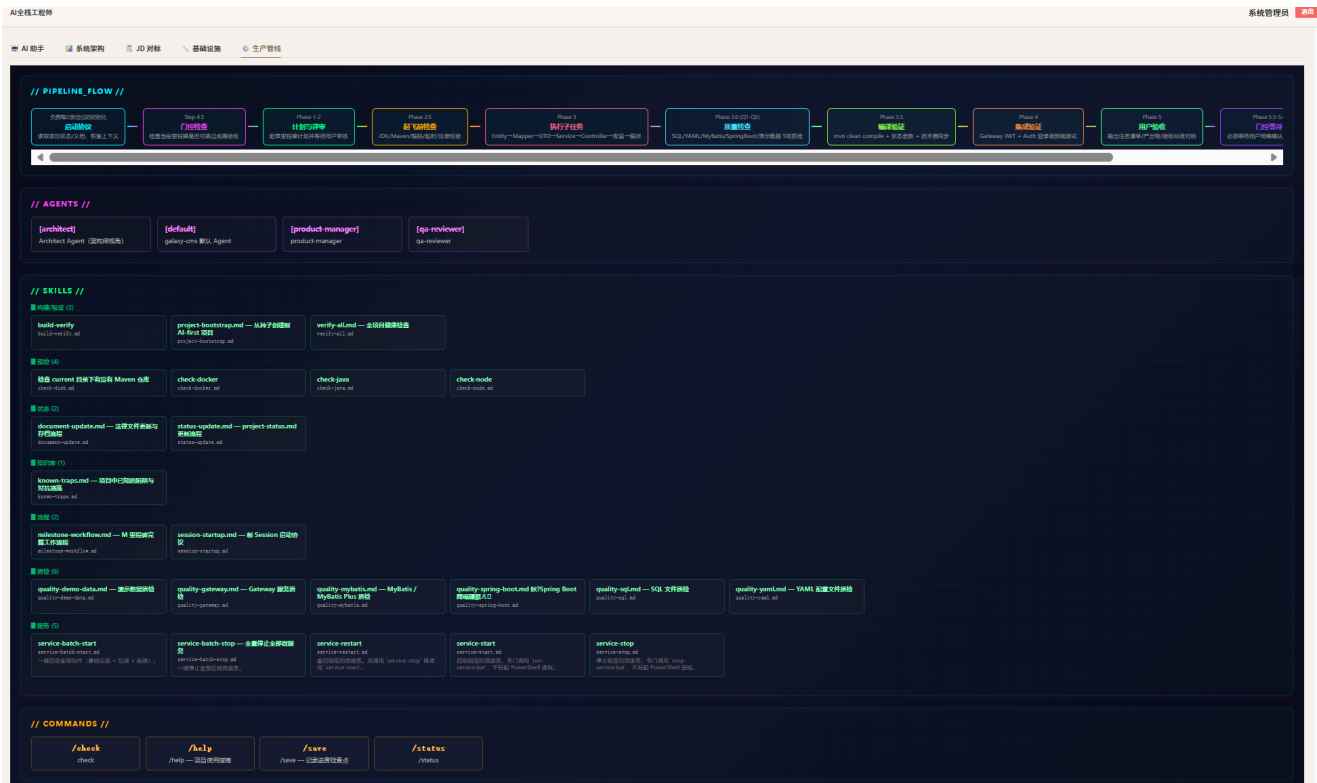
测试 RabbitMQ 消息

CI/CD 构建状态

查看流水线 ->

GitHub: wind-laughing/traffic-demo

生产管线



七、AI-first开发机制

11阶管线

Session Init → Gate Check → Plan(人类审批) → Preflight Check → Execute → Quality Check → Compile Verify → Integration Verify → User Accept → Gate → Archive

AI负责初稿生成、代码执行、文档沉淀；人类负责架构判断、质量把关、门控验收。

项目治理数据

指标	数据
总工时	约47小时
里程碑数	18个
后端微服务	8个
后端Java源文件	120+个
前端Vue源文件	60+个
数据库表	14张（MySQL）+ 全文索引（PostgreSQL）
总Bug修复	14项（M15集中修复）
Git提交规范	Conventional Commits + 里程碑Tag

八、总结：从老系统到AI-first模式的升级路径

本系统证明的核心结论不是"AI能写代码"，而是：

老系统的升级，核心障碍不是技术债，是编程模式。换了编程模式，老系统才能跟上业务变化的速度。

升级路径建议：

阶段	做什么	预期效果
第一阶段	引入AI编码工具，小团队试点	单体功能开发效率提升2-3倍
第二阶段	沉淀Agent角色/Skill/质量门禁体系	团队协作标准化，新人上手周期缩短
第三阶段	逐步重构关键模块，建立规约体系	老系统可维护性系统提升
第四阶段	全链路AI-first开发，建立企业级工程能力	需求响应速度从周级降到天级

每个阶段都需要"既懂老系统又懂AI工具"的人来带队——这正是AI全栈工程师的核心价值。